

[Meus Cursos](#)[MBA em Engenharia de Software com IA](#)[Desafios Técnicos - MBA IA](#)

Ingestão e Busca Semântica com LangChain e Postgres

Objetivo

Você deve entregar um software capaz de:

- Ingestão: Ler um arquivo PDF e salvar suas informações em um banco de dados PostgreSQL com extensão pgVector.
- Busca: Permitir que o usuário faça perguntas via linha de comando (CLI) e receba respostas baseadas apenas no conteúdo do PDF.

Exemplo no CLI

Faça sua pergunta:

PERGUNTA: Qual o faturamento da Empresa SuperTechIABrazil?

RESPOSTA: O faturamento foi de 10 milhões de reais.

Perguntas fora do contexto:

PERGUNTA: Quantos clientes temos em 2024?

RESPOSTA: Não tenho informações necessárias para responder sua pergunta.

Tecnologias obrigatórias

- Linguagem: Python
- Framework: LangChain
- Banco de dados: PostgreSQL + pgVector
- Execução do banco de dados: Docker & Docker Compose (docker-compose fornecido no repositório de exemplo)



Pacotes recomendados

- Split: `from langchain_text_splitters import RecursiveCharacterTextSplitter`
 - Embeddings (OpenAI): `from langchain_openai import OpenAIEmbeddings`
 - Embeddings (Gemini): `from langchain_google_genai import GoogleGenerativeAIEmbeddings`
 - PDF: `from langchain_community.document_loaders import PyPDFLoader`
 - Ingestão: `from langchain_postgres import PGVector`
 - Busca: `similarity_search_with_score(query, k=10)`
-

OpenAI

- Crie uma API Key da OpenAI.
 - Você vai precisar de um modelo de embeddings e de um modelo de LLM para responder. Consulte a documentação oficial da OpenAI para ver os modelos disponíveis.
-

Gemini

- Crie uma API Key da Google.
- Você vai precisar de um modelo de embeddings e de um modelo de LLM para responder. Consulte a documentação oficial do Google para ver os modelos disponíveis.

Os limites de requisições gratuitas dos modelos podem mudar com frequência. Para informações atualizadas, consulte a documentação oficial do Google.

Escolha dos modelos

Este desafio não fixa modelos. Nomes e versões mudam com frequência e alguns são descontinuados, então faz parte do desafio consultar a documentação oficial do provedor que você escolher, ver quais modelos estão disponíveis no momento e selecionar os que atendem ao objetivo. Para o volume deste desafio, os modelos mais leves e baratos de cada provedor são suficientes.

Atenção: modelos de embedding diferentes geram vetores com dimensões diferentes. A tabela de vetores é criada na primeira ingestão, já com a dimensão do modelo que você escolheu. Se você trocar de modelo de embeddings depois disso, a ingestão passa a falhar por incompatibilidade de dimensão. Nesse caso é responsabilidade sua apagar a collection existente (ou o volume do banco) e refazer a ingestão do zero com o novo modelo.

Requisitos

1. Ingestão do PDF

- O PDF deve ser dividido em chunks de 1000 caracteres com overlap de 150.
- Cada chunk deve ser convertido em embedding.
- Os vetores devem ser armazenados no banco de dados PostgreSQL com pgVector.

2. Consulta via CLI

Criar um script Python para simular um chat no terminal.

Passos ao receber uma pergunta:

- Vetorizar a pergunta.
- Buscar os 10 resultados mais relevantes (k=10) no banco vetorial.
- Montar o prompt e chamar a LLM.
- Retornar a resposta ao usuário.

Prompt a ser utilizado:

CONTEXT0:

{resultados concatenados do banco de dados}

REGRAS:

- Responda somente com base no CONTEXT0.
- Se a informação não estiver explicitamente no CONTEXT0, responda:

"Não tenho informações necessárias para responder sua pergunta."

- Nunca invente ou use conhecimento externo.
- Nunca produza opiniões ou interpretações além do que está escrito.

EXEMPLOS DE PERGUNTAS FORA DO CONTEXT0:

Pergunta: "Qual é a capital da França?"

Resposta: "Não tenho informações necessárias para responder sua pergunta."

Pergunta: "Quantos clientes temos em 2024?"

Resposta: "Não tenho informações necessárias para responder sua pergunta."

Pergunta: "Você acha isso bom ou ruim?"

Resposta: "Não tenho informações necessárias para responder sua pergunta."

PERGUNTA DO USUÁRIO:
{pergunta do usuário}

RESPONDA A "PERGUNTA DO USUÁRIO"

Estrutura obrigatória do projeto

Faça um fork do repositório para utilizar a estrutura abaixo:

<https://github.com/devfullcycle/mba-ia-desafio-ingestao-busca>

```
├─ docker-compose.yml
├─ requirements.txt      # Dependências
├─ .env.example          # Template das variáveis de
ambiente
├─ src/
|   ├─ ingest.py         # Script de ingestão do PDF
|   ├─ search.py         # Script de busca
|   └─ chat.py           # CLI para interação com usuário
├─ document.pdf          # PDF para ingestão
└─ README.md             # Instruções de execução
```

VirtualEnv para Python

Crie e ative um ambiente virtual antes de instalar dependências:

```
python3 -m venv venv
source venv/bin/activate
```

Ordem de execução

1. Subir o banco de dados:

```
docker compose up -d
```

2. Executar ingestão do PDF:

```
python src/ingest.py
```

3. Rodar o chat:

```
python src/chat.py
```

Entregável

Repositório público no GitHub contendo todo o código-fonte e README com instruções claras de execução do projeto.

Área de entrega

Insira a URL completa do repositório público, incluindo **https://**.

Insira o link do repositório

Enviar

Iniciar

Aulas

Desafios



Iniciar